



HOPR v4 Update Audit

Gnosis Ltd - Report by Côme du Crest

2026-02-14



Table of contents

- Table of contents
- HOPR v4 Update Audit
 - Scope
 - Context
 - Status
 - Legal Information And Disclaimer
- Issues
 - [Info] Memory safety of HoprNodeStakeFactory is frail
 - Optimisations and miscellaneous

HOPR v4 Update Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

The scope includes selected contracts within `hoprnet/contracts` as of commit `0x8105465`.

Contracts in scope are:

```
1 |-- Announcements.sol # Announce nodes' multiaddress and key-bindings
2 |-- Channels.sol # payment channels between nodes in the HOPR network
3 |-- Crypto.sol # crypto primitives
4 |-- Ledger.sol # snapshot-based indexing of Hopr Channels
5 |-- MultiSig.sol # abstraction of interaction between nodes and Safes in the
   HOPR network
6 |-- TicketPriceOracle.sol # oracle for network ticket price
7 |-- WinningProbabilityOracle.sol # oracle for network minimum winning
   probability
8 |-- node-stake
9 |   |-- migration
10 |       |-- NodeSafeMigration.sol # registry for nodes and Safes in the HOPR
   network
11 |   |-- NodeSafeRegistry.sol # ledger to register the association of nodes and
   safes
12 |   |-- NodeStakeFactory.sol # factory contract to deploy Safe and node
   management Module for node runners
13 |   |-- permissioned-module
14 |       |-- CapabilityPermissions.sol # library for capability management of
   node management Module
15 |       |-- NodeManagementModule.sol # implementation logics for node
   management Module
16 |-- utils
17 |   |-- EnumerableKeyBindingSet.sol # enumerable sets for types related to key-
   bindings
```

Context

This audit follows a previous audit on 2024-08-26 and changes to the contracts are limited. The contracts are not assumed to be upgraded and storage layout discrepancies are not considered issues.

Status

The report has been sent to the core developer.

Legal Information And Disclaimer

1. This report is based solely on the information provided by HOPR (the “Company”), with the assumption that the information provided to Gnosis is authentic, accurate, complete, and not misleading as of the date of this report. Gnosis has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. Gnosis is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, Gnosis does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. Gnosis disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely our views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. Gnosis is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. This report is addressed exclusively to the Company, and except with prior written consent from Gnosis, it may not be shared, disclosed, transmitted, or relied upon by any other person or entity for any purpose.
8. To the fullest extent permitted by applicable laws, Gnosis disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Info] Memory safety of HoprNodeStakeFactory is frail

Summary

The functions of `HoprNodeStakeFactory`: `_deploySafeAndModuleAndIncludeNodes()` and `_deploySafeAndModule()` append the address of the factory to the memory array `admins` to set itself as a temporary admin to set up the safe. It does so in assembly by increasing the length of the array and storing the address at the end. The memory safety of this operation is not guaranteed and may work in the current compilation setting and function call order but break in the future.

Vulnerability Detail

The function `tokensReceived` allocates the memory array `admins` before calling either `_deploySafeAndModule()` or `_deploySafeAndModuleAndIncludeNodes()` which function in a similar way with regards to the `admins` array:

```
1      function tokensReceived(  
2          address, // operator not needed  
3          address from,  
4          address to,  
5          uint256 amount,  
6          bytes calldata userData,  
7          bytes calldata // operatorData not needed  
8      )  
9      external  
10     override  
11     {  
12         ...  
13         (bytes32 functionIdentifier, uint256 nonce, bytes32 defaultTarget,  
14             address[] memory admins) =  
15             abi.decode(userData, (bytes32, uint256, bytes32, address[]));  
16         address payable safeProxy;  
17         // Ensure the function selector matches the expected value for this  
18         // operation  
19         if (functionIdentifier == DEPLOYSAFEMODULE_FUNCTION_IDENTIFIER) {  
20             // Deploy Safe and module proxies  
21             (, safeProxy) = _deploySafeAndModule(  
22                 nonce,  
23                 defaultTarget,  
24                 from,  
25                 defaultHoprNetwork.tokenAddress,  
26                 defaultHoprNetwork.defaultTokenAllowance,  
27                 admins  
28             );
```

```
28         } else if (functionIdentifier ==
29             DEPLOYSAFEANDMODULEANDINCLUDENODES_IDENTIFIER) {
30             // Deploy Safe and module proxies using the clone function
31             (, safeProxy) = _deploySafeAndModuleAndIncludeNodes(
32                 nonce,
33                 defaultTarget,
34                 from,
35                 defaultHopNetwork.tokenAddress,
36                 defaultHopNetwork.defaultTokenAllowance,
37                 admins
38             );
39         } else {
40             revert InvalidFunctionSelector();
41         }
42     }
```

The function `_deploySafeAndModule()` resizes the `admins` array in assembly to append itself before calling `_deploySafe()`:

```
1     function _deploySafeAndModule(
2         uint256 nonce,
3         bytes32 defaultTarget,
4         address caller,
5         address tokenAddress,
6         uint256 approveAmount,
7         address[] memory admins
8     )
9     internal
10    validateAdmins(admins)
11    returns (address, address payable)
12    {
13        // Temporarily add the factory address as an owner
14        assembly {
15            let len := mload(admins)
16            mstore(admins, add(len, 1))
17            mstore(add(admins, mul(0x20, add(len, 1))), address())
18        }
19
20        // 1. Prepare safe initializer data
21        address payable safeProxyAddr = _deploySafe(admins, nonce);
22
23        ...
24    }
```

This only works because there is no memory allocation in between the first allocation of `admins`, it being resized, and the last word of `admins` being used in `_deploySafe()`. It also relies on the current default behaviour of solc to avoid using memory for optimization / to avoid stack too deep when unsafe assembly is used in a contract.

Impact

If the contract comes to change or the compilation settings are changed, the deployment of safes may break as some variable in memory may be overwritten or the factory may fail to set itself as a temporary admin as the last word of the `admins` array may be overwritten.

Code Snippets

<https://github.com/hoprnet/contracts/blob/8105465a7784b042cff5a34ddd5004041fa9c9f6/ethereum/contracts/src/node-stake/NodeStakeFactory.sol#L389>

<https://github.com/hoprnet/contracts/blob/8105465a7784b042cff5a34ddd5004041fa9c9f6/ethereum/contracts/src/node-stake/NodeStakeFactory.sol#L449>

Recommendation

Make the user of the factory (through the HOPR Token) encode the address of the factory as the last element of the `admins` array in the `userData` of `tokensReceived()`. That way the factory only need to verify that it is the last element of the `admins` array instead of resizing it.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

HoprLedger does not use efficient hashing

This contract update introduces Solady's `EfficientHashLib` to save gas for hashing. However, the library is not used in `HoprLedger.indexEvent()` which is an extensively used function in the protocol.

I am not entirely sure under which circumstances the `EfficientHashLib` is more efficient than solidity's classical `keccak256()` so I am not sure whether it is useful here but it is worth noting.

<https://github.com/hoprnet/contracts/blob/8105465a7784b042cff5a34ddd5004041fa9c9f6/ethereum/contracts/src/Ledger.sol#L120>

HoprChannels version is still 2.0.0

The contract `HoprChannels` has been updated but the constant `VERSION` (which should probably have been called `CHANNELS_VERSION`) remain `2.0.0` as in the previous contract version.

<https://github.com/hoprnet/contracts/blob/8105465a7784b042cff5a34ddd5004041fa9c9f6/ethereum/contracts/src/Channels.sol#L191>

HoprAnnouncements does not use storage gaps

The contract `HoprAnnouncements` is UUPSUpgradeable and inherits from different contracts including `HoprMultiSig` and `HoprLedger` which use storage but do not provide for a storage gap.

If contracts inherited by `HoprAnnouncements` need updating it will not be possible to add storage slots without having storage layout incompatibilities. It will also be impossible to inherit a new contract that uses storage slots.

It is recommended to add a storage gap in contracts that may have their storage use updated in the future.

<https://github.com/hoprnet/contracts/blob/8105465a7784b042cff5a34ddd5004041fa9c9f6/ethereum/contracts/src/Announcements.sol#L104-L110>

<https://github.com/hoprnet/contracts/blob/8105465a7784b042cff5a34ddd5004041fa9c9f6/ethereum/contracts/src/MultiSig.sol#L30>

<https://github.com/hoprnet/contracts/blob/8105465a7784b042cff5a34ddd5004041fa9c9f6/ethereum/contracts/src/Ledger.sol#L37>